Algoritmos y diagramas de flujo

Nombre: _____ Fecha: _____

Qué vas a aprender

- Definir algoritmo y reconocer sus características
- Conocer los símbolos de los diagramas de flujo
- Representar algoritmos con estructuras secuenciales, condicionales y repetitivas
- Escribir algoritmos en pseudocódigo y seguirlos con una traza

Para pensar

¿Cómo le explicarías a alguien que nunca ha visto un cajero automático los pasos exactos para sacar dinero, incluyendo qué hacer si se equivoca de PIN?

Explicación

1.1 ¿Qué es un algoritmo?

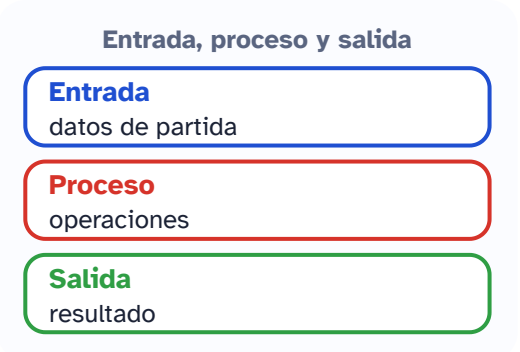
Un **algoritmo** es un conjunto finito y ordenado de instrucciones precisas que resuelve un problema. La palabra viene del matemático persa **al-Juarismi** (siglo IX), que escribió métodos paso a paso para hacer cálculos.

Todo algoritmo tiene tres partes:

- **Entrada**: los datos que recibe (por ejemplo, dos números).
- **Proceso**: las operaciones que realiza (sumarlos).
- **Salida**: el resultado que devuelve (mostrar la suma).

Un mismo problema puede resolverse con algoritmos distintos; uno es **mejor** si da el resultado correcto con menos pasos o menos recursos.

Todo algoritmo tiene tres partes: los datos de **entrada** (lo que necesita para empezar), el **proceso** (las operaciones) y la **salida** (el resultado). Y debe cumplir cuatro propiedades: ser **preciso** (cada paso tiene un único significado), **finito** (termina tras un número limitado de pasos), **ordenado** y **definido** (con los mismos datos de entrada da siempre el mismo resultado).



1.2 Símbolos del diagrama de flujo

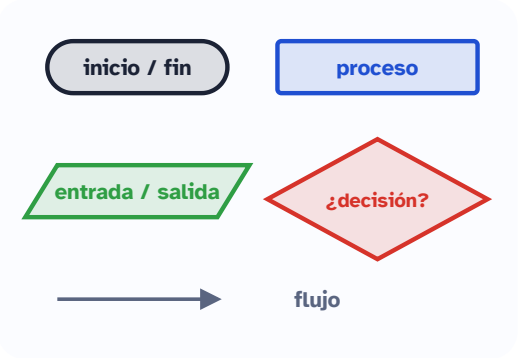
Un **diagrama de flujo** representa gráficamente un algoritmo. Usa símbolos normalizados unidos por **flechas** que indican el orden:

Símbolo	Forma	Significado
Inicio / fin	óvalo	dónde empieza y termina

Símbolo	Forma	Significado
Proceso	rectángulo	una acción o cálculo: `suma ← a + b`
Entrada / salida	paralelogramo	leer datos o mostrar resultados
Decisión	rombo	una pregunta con respuesta sí / no
Flecha	línea con punta	sentido del flujo

Reglas: un solo inicio, al menos un fin, las flechas no se cruzan y cada rombo tiene dos salidas.

Los diagramas de flujo se leen de arriba abajo y de izquierda a derecha. Para que sean claros conviene escribir dentro de cada símbolo frases muy cortas (leer edad, área ← b · h), etiquetar las dos salidas de cada rombo con «sí» y «no» y, si el diagrama es largo, usar **conectores** (pequeños círculos con una letra) para continuar en otra parte de la hoja sin que las flechas se crucen.

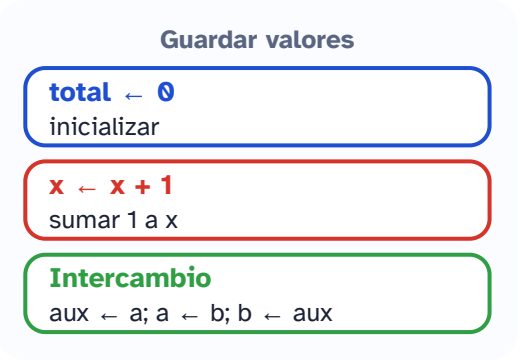


1.3 Variables y asignación

Los algoritmos trabajan con datos que se guardan en **variables**: cajas con un nombre (nota, edad, total) que contienen un valor que puede cambiar. Para dar un valor a una variable se usa la **asignación**, que en pseudocódigo se escribe con una flecha: **total ← 0** se lee «total toma el valor 0». La flecha indica que el valor de la derecha se guarda en la variable de la izquierda.

La asignación no es una igualdad matemática. La instrucción **x ← x + 1** no tiene sentido en matemáticas, pero en un algoritmo significa: «toma el valor actual de x, súmale 1 y guarda el resultado de nuevo en x». Si x valía 4, ahora vale 5. Al asignar un valor nuevo, el anterior se pierde. Por eso, para intercambiar los valores de dos variables a y b hace falta una tercera variable auxiliar: aux ← a; a ←

b; b ← aux. Una variable debe tener siempre un valor inicial antes de usarse: es lo que se llama **inicializar**.



2.1 Estructura secuencial

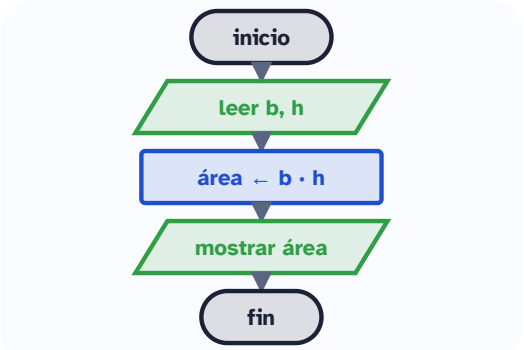
La estructura más sencilla es la **secuencia**: las instrucciones se ejecutan una detrás de otra, siempre en el mismo orden.

Ejemplo: calcular el área de un rectángulo.

1. Inicio.
2. Leer la base `b` y la altura `h`.
3. Calcular `área ← b · h`.
4. Mostrar `área`.
5. Fin.

La flecha ← significa «guarda el resultado en». Si `b = 5` y `h = 3`, el programa muestra 15.

En una secuencia el **orden importa**: si se calcula el área antes de leer la base y la altura, el resultado es un disparate. Por eso, al diseñar, hay que preguntarse qué datos necesita cada paso y asegurarse de que ya se han obtenido. Otro ejemplo: para convertir grados Celsius a Fahrenheit, primero se lee c, después se calcula f ← c · 1,8 + 32 y, por último, se muestra f.


inicio
leer b, h
área ← b · h
mostrar área
fin

2.2 Estructura condicional

Muchas veces el algoritmo debe **decidir**. La estructura **condicional** (o selectiva) evalúa una condición y sigue un camino u otro:

- **Si… entonces**: si se cumple la condición, hace algo; si no, sigue.
- **Si… entonces… si no**: dos caminos alternativos.

Ejemplo: decidir si un alumno aprueba.

```
leer nota
si nota ≥ 5 entonces
    mostrar "aprobado"
si no
    mostrar "suspenso"
fin si
```

Las condiciones usan operadores de comparación: =, ≠, <, >, ≤, ≥, y se pueden combinar con **y**, **o**, **no**.

Las condiciones se escriben con **operadores de comparación**: igual (=), distinto (≠), mayor (>), menor (<), mayor o igual (≥) y menor o igual (≤). El resultado de una condición siempre es **verdadero** o **falso**. Hay que leer con cuidado el enunciado: «aprueba con un 5» se escribe nota ≥ 5, no nota > 5.

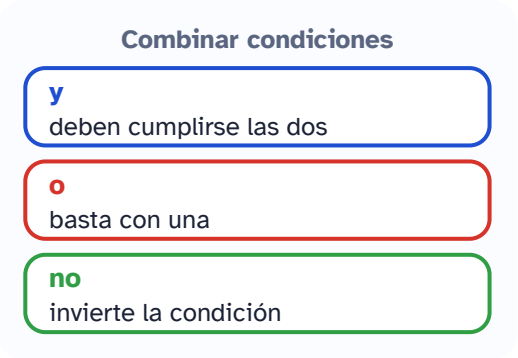


2.3 Condiciones anidadas y múltiples

Cuando hay más de dos posibilidades, las condiciones se pueden **anidar**: poner un condicional dentro de otro. Por ejemplo, para convertir una nota numérica en calificación: si nota < 5, «insuficiente»; si no, si nota < 6, «suficiente»; si no, si nota < 7, «bien»; si no, si nota < 9, «notable»; si no, «sobresaliente». En el

diagrama de flujo aparecen varios rombos en cascada, cada uno en la salida «no» del anterior.

Las condiciones también se pueden **combinar** con operadores lógicos. Con **y** deben cumplirse las dos: «si edad ≥ 12 y altura ≥ 140, puede subir a la atracción». Con **o** basta con una: «si es sábado o es domingo, no hay clase». Con **no** se invierte la condición. Muchos lenguajes tienen además una estructura de **selección múltiple** (según… caso) que elige entre muchas opciones según el valor de una variable, por ejemplo el día de la semana.

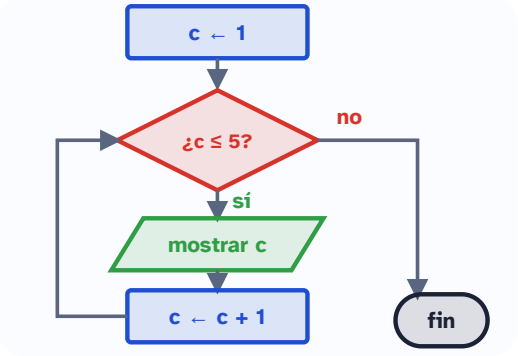


2.4 Estructura repetitiva (bucles)

Un **bucle** repite un grupo de instrucciones. Hay dos tipos básicos:

- **Repetir N veces** (bucle *para*): se sabe de antemano cuántas veces. «Repite 10 veces: salta».
- **Mientras** se cumpla una condición: se repite hasta que deja de cumplirse. «Mientras no llegues a la puerta: da un paso».

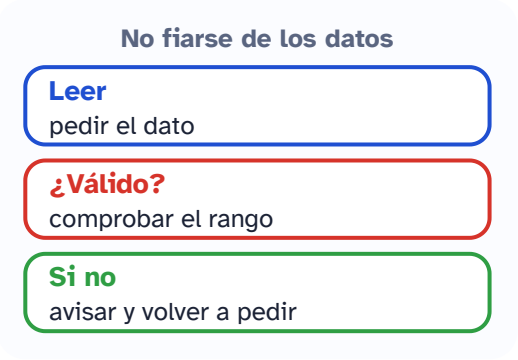
En el diagrama de flujo, el bucle se ve como una flecha que vuelve hacia atrás hasta un rombo. ¡Cuidado con los **bucles infinitos**! Si la condición nunca cambia, el programa no termina y deja de ser un algoritmo.



2.5 Validar los datos de entrada

Un buen algoritmo no se fía de los datos que recibe. Si pide la edad y el usuario escribe −5 o 300, el resultado será absurdo. Por eso se **validan** los datos de entrada con un bucle: se pide el dato y, **mientras** no sea válido, se muestra un mensaje de error y se vuelve a pedir. Ejemplo en pseudocódigo: leer edad; mientras edad < 0 o edad > 120: mostrar «Edad no válida», leer edad; fin mientras.

Otra variante es el bucle **repetir… hasta que**, que ejecuta las instrucciones al menos una vez y comprueba la condición al final: repetir: leer PIN; hasta que PIN sea correcto o intentos = 3. La validación se usa en todos los formularios: comprobar que un correo lleva @, que una contraseña tiene suficientes caracteres o que una fecha existe. Evita errores, hace los programas más robustos y es también una medida de **seguridad**.

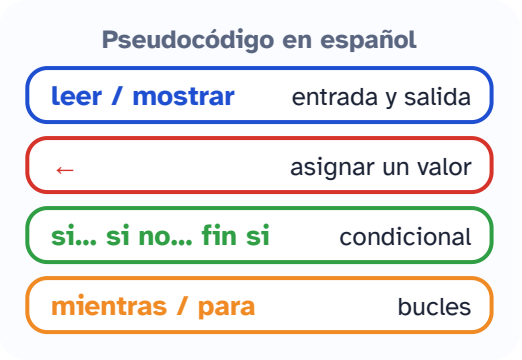


3.1 Pseudocódigo

El **pseudocódigo** describe un algoritmo con palabras del lenguaje natural, pero con la estructura de un programa. Es un paso intermedio entre la idea y el código real: no depende de ningún lenguaje concreto.

```
inicio
  suma ← 0
  para i desde 1 hasta 10
      suma ← suma + i
  fin para
  mostrar suma
fin
```

Este algoritmo suma los números del 1 al 10 y muestra 55. Las palabras clave (**si**, **mientras**, **para**, **leer**, **mostrar**) y la **sangría** ayudan a ver qué instrucciones están dentro de cada estructura.



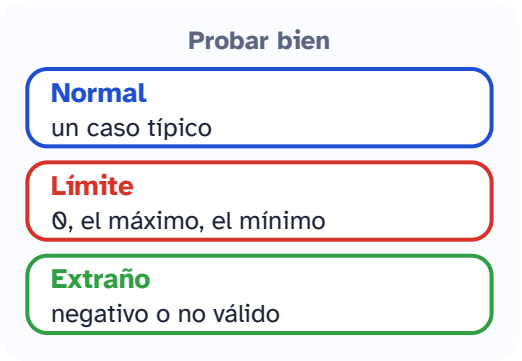
3.2 La traza: comprobar un algoritmo

Hacer una **traza** es seguir el algoritmo a mano, paso a paso, anotando el valor de cada variable. Es la mejor forma de comprobar si funciona y de encontrar errores antes de programarlo.

Paso	c	¿c ≤ 3?	Se muestra
1	1	sí	1
2	2	sí	2
3	3	sí	3
4	4	no	fin

La tabla es la traza del bucle «mientras c ≤ 3: mostrar c; c ← c + 1», empezando con `c = 1`. Si olvidáramos `c ← c + 1`, la traza mostraría 1, 1, 1... sin fin: un bucle infinito.

Para que una traza sea útil hay que elegir bien los **datos de prueba**: un caso normal, los casos límite (el primer y el último valor posibles, como 0 o el máximo) y algún caso extraño o no válido (un número negativo, un texto en lugar de un número). Muchos errores solo aparecen en los casos límite, como confundir «<» con «≤».



4.1 Algoritmos famosos

Algunos algoritmos cambiaron el mundo:

- **Búsqueda binaria**: para encontrar una palabra en un diccionario, abres por la mitad y descartas la mitad que no sirve. Con 1.000 palabras bastan unos 10 pasos.
- **Algoritmos de ordenación**: ordenan listas (de menor a mayor, alfabéticamente). Los hay lentos y rápidos.
- **Ruta más corta** (Dijkstra): lo usan los navegadores GPS.
- **PageRank**: ordenó los resultados del primer buscador de Google.

Hoy los algoritmos deciden qué vídeos ves o qué anuncios aparecen. Entenderlos ayuda a usarlos de forma crítica.

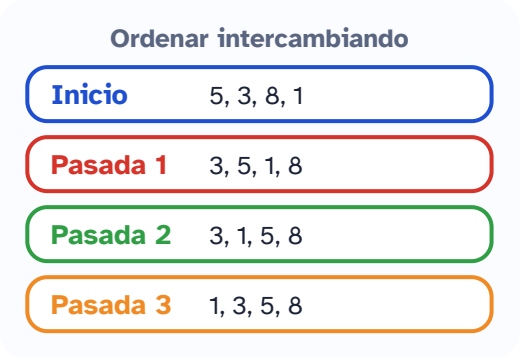
También los algoritmos de **cifrado** protegen cada compra y cada mensaje que enviamos, y los de **compresión** reducen el tamaño de fotos, música y vídeos (JPG, MP3, MP4) aprovechando los patrones que se repiten. La mayoría de ellos se basan en las mismas tres estructuras que aprendemos en este tema, combinadas de forma ingeniosa.



4.2 Ordenar datos: el método de la burbuja

Ordenar es una de las tareas más frecuentes de los ordenadores: los contactos por nombre, los productos por precio, los resultados por puntuación. Uno de los algoritmos más sencillos es el **método de la burbuja**: se recorre la lista comparando cada elemento con el siguiente y, si están en el orden incorrecto, se intercambian. Al terminar la primera pasada, el mayor ha «subido» al final como una burbuja. Se repiten las pasadas hasta que en una no haya ningún intercambio.

Ejemplo con 5, 3, 8, 1. Primera pasada: 5 y 3 se intercambian (3, 5, 8, 1); 5 y 8 no; 8 y 1 se intercambian (3, 5, 1, 8). Segunda pasada: 3, 1, 5, 8. Tercera: 1, 3, 5, 8. Cuarta: ningún cambio, terminado. Es fácil de entender pero lento con muchos datos; existen algoritmos más eficientes, como la **ordenación por mezcla** (merge sort) o **quicksort**, que dividen la lista en partes, las ordenan y las combinan.



4.3 Herramientas para diseñar algoritmos

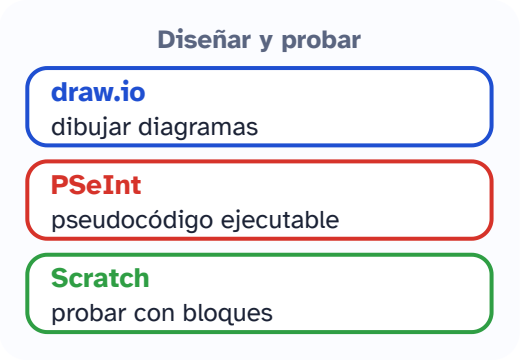
Puedes dibujar diagramas de flujo a mano o con programas gratuitos como **draw.io** (diagrams.net). Otras herramientas permiten además **ejecutarlos**:

- **PSeInt**: escribe pseudocódigo en español, lo ejecuta y genera el diagrama de flujo automáticamente.
- **Flowgorithm**: construyes el diagrama y lo ejecuta paso a paso.
- **Scratch**: los bloques encajables son, en el fondo, un pseudocódigo visual.

El orden de trabajo recomendado es: entender el problema → diseñar el algoritmo → hacer una traza → programarlo → probarlo.

Sea cual sea la herramienta, el método es el mismo: entender el problema, escribir el algoritmo a mano o en pseudocódigo, dibujar el diagrama, hacer la traza con varios datos de prueba y solo entonces programarlo. Guardar el

diagrama junto al programa ayuda a otras personas (y a uno mismo semanas después) a entender cómo funciona.



4.4 Del algoritmo al programa

Una vez diseñado y comprobado, el algoritmo se escribe en un **lenguaje de programación**. Las estructuras son las mismas en todos; solo cambia cómo se escriben. En Scratch, «leer» es el bloque «preguntar y esperar», «mostrar» es «decir», la asignación es «fijar variable a», el condicional es «si... entonces... si no» y los bucles son «repetir» y «repetir hasta que». En **Python**, el mismo algoritmo se escribe con palabras en inglés: input() para leer, print() para mostrar, = para asignar, if... else para el condicional y for o while para los bucles.

Por ejemplo, «leer edad; si edad ≥ 18 mostrar "Puede votar" si no mostrar "No puede votar"» en Python sería: edad = int(input("Edad: ")), luego if edad >= 18: print("Puede votar"), y else: print("No puede votar"). En Python la **sangría** (los espacios a la izquierda) indica qué instrucciones están dentro del condicional o del bucle. Diseñar primero el algoritmo hace que programar sea mucho más rápido y con menos errores, sea cual sea el lenguaje.



Las tres estructuras de control

Secuencia	una instrucción detrás de otra
Condicional	si… entonces… si no (rombo con sí / no)
Repetitiva	repetir N veces · mientras se cumpla
Pseudocódigo	palabras + estructura de programa
Traza	seguir a mano el valor de cada variable

Ejercicios

1. **Indica la entrada, el proceso y la salida de un algoritmo que calcula el perímetro de un cuadrado.**

2. **Dibuja el diagrama de flujo que lee dos números y muestra el mayor.**

3. **Escribe en pseudocódigo un algoritmo que diga si un número es par.**

4. **¿Cuántas veces se ejecuta el interior de «para i desde 3 hasta 8»?**

5. **¿Qué muestra: a ← 2; b ← a + 3; a ← b · 2; mostrar a?**

6. **Haz la traza de: s ← 0; para i de 1 a 5: s ← s + 2; mostrar s.**

7. **Corrige el bucle infinito: c ← 1; mientras c < 5: mostrar c.**

8. **¿Cuántos pasos necesita como máximo la búsqueda binaria en una lista ordenada de 16 elementos?**

9. **Diseña el algoritmo del cajero automático con un máximo de 3 intentos de PIN.**

10. **Escribe el algoritmo de un juego en el que el ordenador piensa un número del 1 al 10 y el jugador lo adivina con pistas «mayor» o «menor».**

11. **¿Qué significa «x ← x + 2»?**

12. **¿Qué es inicializar una variable?**

13. **Escribe la condición para que una persona pueda entrar a una piscina si tiene 14 años o más o va acompañada de un adulto.**

14. **¿Para qué sirve validar los datos de entrada?**

15. **¿Cómo se escriben «leer» y «mostrar» en Python?**

16. **¿Qué tipos de datos de prueba conviene usar en una traza?**

17. **Escribe en pseudocódigo un algoritmo que lea una contraseña y la pida de nuevo mientras tenga menos de 8 caracteres.**

18. **Haz la traza: x ← 1; mientras x < 20: x ← x · 3; mostrar x.**

19. **Ordena con burbuja: 9, 7, 5, 3. ¿Cuántos intercambios hay en total?**

20. **Escribe un algoritmo que diga si un año es bisiesto (divisible entre 4, salvo los divisibles entre 100 que no lo sean entre 400).**

21. **Traduce a Python: leer n; si n mod 2 = 0 mostrar «par» si no mostrar «impar».**

22. **Diseña con selección múltiple un algoritmo que muestre el nombre del día de la semana a partir de un número del 1 al 7.**

23. **¿Qué diferencia hay entre «mientras» y «repetir... hasta que»?**

24. **Escribe un algoritmo que lea tres números y muestre el mayor usando condiciones anidadas.**

25. **¿Qué hace la sangría en Python?**

26. **Diseña el diagrama de flujo de un juego de adivinar un número del 1 al 100 en el que el ordenador dice «mayor» o «menor» y cuenta los intentos.**

27. **¿Cuántas pasadas necesita como máximo el método de la burbuja para ordenar una lista de n elementos? ¿Por qué es lento con un millón de datos?**

28. **Escribe un algoritmo de cajero que pida el PIN hasta 3 veces con «repetir… hasta que», y bloquee la tarjeta si se falla tres veces.**

29. **Encuentra el error: «suma ← 0; para i de 1 a 10: suma ← i; mostrar suma» debería mostrar 55.**

30. **Diseña y traduce a Python un algoritmo que lea notas hasta que se introduzca –1 y muestre la media.**

Practica

1. **En un diagrama de flujo, el rombo representa...**

   A) un proceso
   B) una decisión
   C) el inicio
   D) una variable

2. **El paralelogramo se usa para...**

   A) decisiones
   B) el fin
   C) entrada o salida de datos
   D) procesos

3. **El rectángulo representa...**

   A) un proceso o cálculo
   B) una pregunta
   C) el inicio
   D) una decisión

4. **Inicio y fin se dibujan con un...**

   A) rombo
   B) triángulo
   C) óvalo
   D) un hexágono

5. **Las tres partes de un algoritmo son...**

   A) inicio, rombo y fin
   B) entrada, proceso y salida
   C) datos, ordenador y pantalla
   D) bucle, sprite y fin

6. **La estructura que ejecuta instrucciones una tras otra es la...**

   A) secuencial
   B) condicional
   C) repetitiva
   D) aleatoria

7. **«Mientras no llegues a la puerta, da un paso» es un...**

A) condicional simple
B) bucle
C) dato de salida
D) operador

8. **Un bucle cuya condición nunca cambia provoca…**

A) un error de sintaxis
B) una salida doble
C) un bucle infinito
D) un error de sintaxis seguro

9. **Seguir a mano un algoritmo anotando las variables es hacer una…**

A) traza
B) compilación
C) abstracción
D) simulación

10. **La palabra algoritmo procede del nombre de…**

A) Arquímedes
B) al-Juarismi
C) Ada Lovelace
D) Newton

11. **«total ← 0» significa…**

A) total es igual a 0 siempre
B) guardar 0 en total
C) comparar total con 0
D) borrar total

12. **Para intercambiar a y b se necesita…**

A) nada más
B) una variable auxiliar
C) dos bucles
D) un rombo

13. **Con el operador «y» la condición se cumple si…**

A) se cumple una
B) se cumplen las dos
C) no se cumple ninguna

D) siempre

14. **Con «o» la condición se cumple si...**

A) se cumplen las dos obligatoriamente

B) se cumple al menos una

C) no se cumple ninguna

D) nunca

15. **Un condicional dentro de otro es un condicional...**

A) anidado

B) infinito

C) secuencial

D) vacío

16. **«Repetir... hasta que» se ejecuta...**

A) nunca

B) al menos una vez

C) solo si la condición es cierta al principio

D) dos veces

17. **El método de la burbuja sirve para...**

A) buscar

B) ordenar

C) sumar

D) dibujar

18. **En Python, print() sirve para...**

A) leer

B) mostrar

C) repetir

D) decidir

19. **En Python, la sangría indica...**

A) el color

B) qué instrucciones están dentro de un bloque

C) comentarios

D) nada

20. **Validar una entrada es...**

A) aceptar cualquier dato

B) comprobar que el dato es correcto

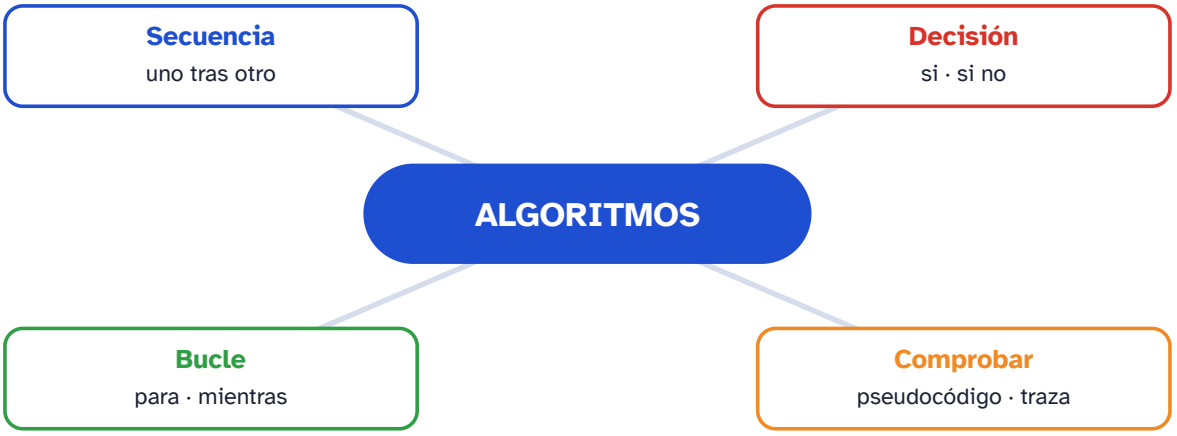
C) borrar datos

D) ordenar

Explícalo con tus palabras

Incluye: Qué es un algoritmo y qué partes tiene; Los símbolos del diagrama de flujo; Las tres estructuras de control con un ejemplo; Cómo se hace una traza y para qué sirve.

Resumen



Glosario

Algoritmo	Conjunto finito y ordenado de instrucciones precisas.
Entrada	Datos que recibe el algoritmo.
Salida	Resultado del algoritmo.
Diagrama de flujo	Representación gráfica con símbolos normalizados.
Variable	Espacio con nombre que guarda un valor.
Asignación	Dar un valor a una variable.
Inicializar	Dar un valor inicial a una variable.
Secuencia	Instrucciones una tras otra.
Condicional	Estructura que elige un camino.
Bucle	Estructura que repite instrucciones.
Bucle infinito	Bucle que nunca termina.
Pseudocódigo	Algoritmo escrito con palabras y estructura de programa.
Traza	Seguimiento a mano del valor de las variables.
Validación	Comprobación de que un dato es correcto.
Ordenación	Colocar datos en un orden.
Sangría	Espacios que indican la estructura en Python.

Tarjetas de repaso

Recórtalas: pregunta delante, respuesta detrás.

Algoritmo

Instrucciones finitas, ordenadas y precisas que resuelven un problema.

Diagrama de flujo

Representación gráfica de un algoritmo con símbolos y flechas.

Rombo

Símbolo de decisión, con salidas sí y no.

Paralelogramo

Símbolo de entrada o salida de datos.

Condicional

Estructura que elige un camino según una condición.

Bucle

Estructura que repite instrucciones.

Pseudocódigo

Algoritmo escrito con palabras y estructura de programa.

Traza

Seguimiento a mano de los valores de las variables.

Asignación

Guardar un valor en una variable: x ← 5.

Condición compuesta

Unir condiciones con y, o, no.

Validación

Repetir la petición mientras el dato no sea válido.

Burbuja

Ordenar intercambiando elementos vecinos.

Soluciones

1. **B)** Pregunta con sí / no.
2. **C)** Leer o mostrar.
3. **A)** Acción.
4. **C)** Terminal.
5. **B)** Recibe, calcula, devuelve.
6. **A)** La más sencilla.
7. **B)** Se repite.
8. **C)** El programa no termina.
9. **A)** Para comprobarlo.
10. **B)** Matemático persa del siglo IX.
11. **B)** Asignación.
12. **B)** aux ← a; a ← b; b ← aux.
13. **B)** Ambas verdaderas.
14. **B)** Basta con una.
15. **A)** Anidado.
16. **B)** Comprueba al final.
17. **B)** Compara e intercambia vecinos.
18. **B)** Salida.
19. **B)** Estructura del programa.
20. **B)** Robustez y seguridad.

Soluciones de los ejercicios

1. Entrada: lado. Proceso: perímetro ← 4 · lado. Salida: mostrar perímetro.
2. Inicio → leer a, b → ¿a > b? → sí: mostrar a; no: mostrar b → fin.
3. leer n; si n mod 2 = 0 entonces mostrar «par» si no mostrar «impar».
4. 6 veces (3, 4, 5, 6, 7 y 8).
5. b = 5, a = 10 → muestra 10.
6. s pasa por 2, 4, 6, 8, 10 → muestra 10.
7. Falta c ← c + 1 dentro del bucle.
8. 4 pasos, porque 16 → 8 → 4 → 2 → 1.
9. intentos ← 0; repetir: leer PIN; intentos ← intentos + 1; hasta PIN correcto o intentos = 3; si correcto, continuar; si no, retener tarjeta.
10. Respuesta abierta; debe incluir un bucle mientras no acierte y una decisión que compare el intento con el número secreto.
11. Toma el valor actual de x, súmale 2 y guarda el resultado en x.
12. Darle un valor inicial antes de usarla, por ejemplo suma ← 0.
13. si edad ≥ 14 o acompañado = sí entonces puede entrar.
14. Para no aceptar datos absurdos o incorrectos (edades negativas, correos sin @), que darían resultados erróneos o fallos de seguridad.
15. input() para leer y print() para mostrar.
16. Un caso normal, los casos límite (0, máximo, mínimo) y algún caso extraño o no válido.
17. leer clave; mientras longitud(clave) < 8: mostrar «Mínimo 8 caracteres»; leer clave; fin mientras; mostrar «Contraseña aceptada».
18. x = 1 → 3 → 9 → 27 (27 ya no es < 20). Muestra 27.
19. P1: 7, 5, 3, 9 (3 intercambios). P2: 5, 3, 7, 9 (2). P3: 3, 5, 7, 9 (1). Total: 6 intercambios.
20. leer año; si (año mod 4 = 0 y año mod 100 ≠ 0) o año mod 400 = 0 entonces mostrar «Bisiesto» si no mostrar «No bisiesto».
21. n = int(input()) if n % 2 == 0: print("par") else: print("impar")
22. leer d; según d: caso 1 mostrar «lunes»; caso 2 «martes»; … caso 7 «domingo»; otro caso mostrar «número no válido».
23. «Mientras» comprueba la condición antes, así que puede no ejecutarse nunca; «repetir… hasta que» la comprueba al final y se ejecuta al menos una vez.
24. leer a, b, c; si a ≥ b y a ≥ c: mostrar a; si no, si b ≥ c: mostrar b; si no: mostrar c.
25. Indica qué instrucciones están dentro de un condicional, un bucle o una función. Una sangría incorrecta cambia el programa o da error.
26. Inicio → secreto ← aleatorio(1, 100); intentos ← 0 → leer n → intentos ← intentos + 1 → ¿n = secreto? sí: mostrar «¡Acertaste en», intentos → fin. No: ¿n < secreto? sí: mostrar «mayor»; no: «menor» → volver a leer n.
27. Como máximo n – 1 pasadas, cada una con hasta n – 1 comparaciones: del orden de n^2 comparaciones. Con un millón serían un billón de comparaciones; por eso se usan algoritmos más rápidos como mergesort.

28. intentos ← 0; repetir: leer pin; intentos ← intentos + 1; si pin ≠ correcto: mostrar «PIN incorrecto»; hasta que pin = correcto o intentos = 3. Si pin = correcto: mostrar menú; si no: mostrar «Tarjeta bloqueada».
29. La asignación sobrescribe en lugar de acumular: suma ← i deja el último valor (10). Debe ser suma ← suma + i. Con la traza se ve: suma vale 1, 2, 3… en lugar de 1, 3, 6…
30. suma = 0; n = 0; nota = float(input()); while nota != -1: suma = suma + nota; n = n + 1; nota = float(input()). Después: if n > 0: print(suma / n). Se valida que haya al menos una nota para no dividir entre 0.